Lab Exercise 18- Volume in Kubernetes Cluster

Name:- Vansh Bhatt

SapID:- 500125395

R. No:- R2142231689

Batch:- DevOps - B1

To:- Hitesh Sharma Sir

Volumes in Kubernetes provide a way to store and share data among containers in a Pod. This lab exercise will guide you through the creation and use of different types of volumes in Kubernetes.

Step 1: Set Up Kubernetes Cluster

Ensure you have access to a Kubernetes cluster. You can use a local setup with Minikube, kind, or use a cloud-based Kubernetes service.

```
vanshssupermac@Vanshs-MacBook-Air-3 Lab18 Kubernetes Cluster Volume % minikube start
😄  minikube v1.38.1 on Darwin 26.3.1 (arm64)
🆕  Kubernetes 1.35.1 is now available. If you would like to upgrade, specify: --kubernetes-version=v
1.35.1
✨  Using the docker driver based on existing profile
👍  Starting "minikube" primary control-plane node in "minikube" cluster
🚜  Pulling base image v0.0.50 ...
🔄  Restarting existing docker container for "minikube" ...
🐳  Preparing Kubernetes v1.35.0 on Docker 29.2.0 ...
🔎  Verifying Kubernetes components...
    ▪ Using image gcr.io/k8s-minikube/storage-provisioner:v5
    ▪ Using image docker.io/kubernetesui/metrics-scraper:v1.0.8
💡  After the addon is enabled, please run "minikube tunnel" and your ingress resources would be avai
lable at "127.0.0.1"
    ▪ Using image registry.k8s.io/ingress-nginx/kube-webhook-certgen:v1.6.7
    ▪ Using image docker.io/kubernetesui/dashboard:v2.7.0
    ▪ Using image registry.k8s.io/ingress-nginx/controller:v1.14.3
    ▪ Using image registry.k8s.io/ingress-nginx/kube-webhook-certgen:v1.6.7
🔎  Verifying ingress addon...
💡  Some dashboard features require the metrics-server addon. To enable all features please run:

        minikube addons enable metrics-server

🌟  Enabled addons: storage-provisioner, dashboard, ingress, default-storageclass
🏄  Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
vanshssupermac@Vanshs-MacBook-Air-3 Lab18 Kubernetes Cluster Volume %
```

Step 2: Create a Pod with an emptyDir Volume

Create a file named **emptydir-pod.yaml** with the following content:

```
apiVersion: v1
kind: Pod
metadata:
  name: test-vol1
spec:
  containers:
    - image: nginx
      name: test-container
      volumeMounts:
        - mountPath: /data
          name: first-volume
  volumes:
    - name: first-volume
      emptyDir: {}
```

```
apiVersion: v1
kind: Pod
metadata:
  name: test-vol1
spec:
  containers:
    - image: nginx
      name: test-container
      volumeMounts:
        - mountPath: /data
          name: first-volume
  volumes:
    - name: first-volume
      emptyDir: {}
~
~
```

In this manifest:

The emptyDir volume is a temporary storage that is created when the Pod is assigned to a node and exists as long as the Pod is running.

Apply the manifest to create the Pod:

```
kubectl apply -f emptydir-pod.yaml
```

```
vanshssupermac@Vanshs-MacBook-Air-3 Lab18 Kubernetes Cluster Volume % kubectl apply -f vbempty-pod.ya
ml
pod/test-vol1 created
vanshssupermac@Vanshs-MacBook-Air-3 Lab18 Kubernetes Cluster Volume %
```

Step 3: Verify the Pod and Volume

Check the status of the Pod:

```
kubectl get pods
```

```
vanshssupermac@Vanshs-MacBook-Air-3 Lab18 Kubernetes Cluster Volume % kubectl get pods
NAME                        READY   STATUS    RESTARTS       AGE
test-vol1                   1/1     Running   0              47s
```

Access the Pod's shell:

```
kubectl exec -it test-vol1 -- bash
```

```
vanshssupermac@Vanshs-MacBook-Air-3 Lab18 Kubernetes Cluster Volume % kubectl exec -it test-vol1 -- b
ash
root@test-vol1:/#
```

Inside the Pod, create a file in the /data directory:

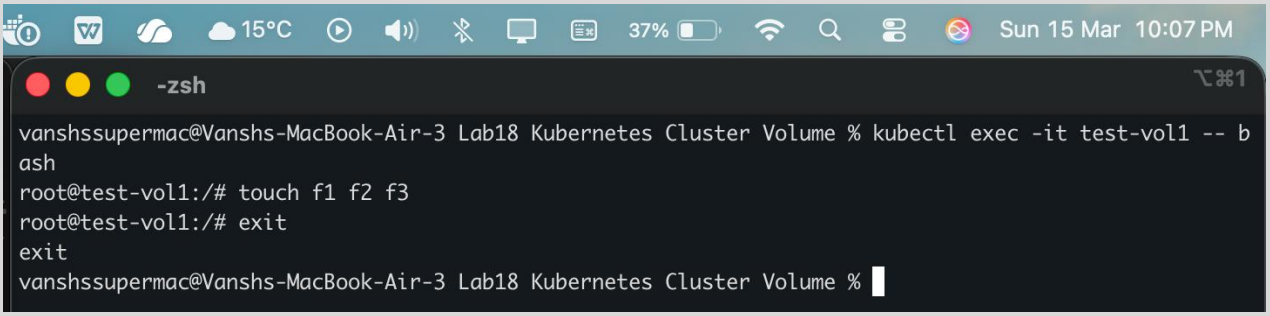
```
touch f1 f2 f3
```

```
vanshssupermac@Vanshs-MacBook-Air-3 Lab18 Kubernetes Cluster Volume % kubectl exec -it test-vol1 -- b
ash
root@test-vol1:/# touch f1 f2 f3
root@test-vol1:/#
```

You should see the text "Hello, Kubernetes!".

Exit the Pod shell:

```
exit
```



Step 4: Create a Pod with a hostPath Volume

Create a file named **hostpath-pod.yaml** with the following content:

```
apiVersion: v1
kind: Pod
metadata:
  name: test-vol2
spec:
  containers:
    - image: coolgourav147/nginx-custom
      name: test-container
      volumeMounts:
        - mountPath: /data
          name: first-volume
  volumes:
    - name: first-volume
      hostPath:
        path: /test
```

```
apiVersion: v1
kind: Pod
metadata:
  name: test-vol2
spec:
  containers:
    - image: coolgourav147/nginx-custom
      name: test-container
      volumeMounts:
        - mountPath: /data
          name: first-volume
  volumes:
    - name: first-volume
      hostPath:
        path: /test
~
~
```

In this manifest:

The hostPath volume mounts a directory from the host into the Pod at /mnt/data.

The busybox container writes the current date to a log file every 5 seconds.

Apply the manifest to create the Pod:

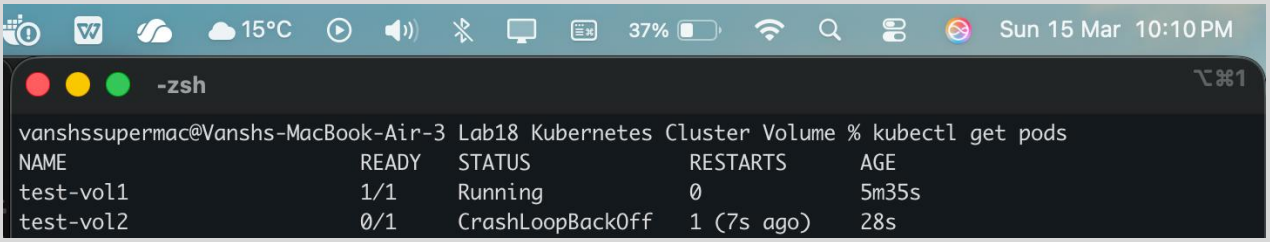
```
kubectl apply -f hostpath-pod.yaml
```

```
vanshssupermac@Vanshs-MacBook-Air-3 Lab18 Kubernetes Cluster Volume % vi hostpath-pod.yaml
vanshssupermac@Vanshs-MacBook-Air-3 Lab18 Kubernetes Cluster Volume % kubectl apply -f hostpath-pod.y
aml
pod/test-vol2 created
vanshssupermac@Vanshs-MacBook-Air-3 Lab18 Kubernetes Cluster Volume %
```

Step 3: Verify the Pod and Volume

Check the status of the Pod:

```
kubectl get pods
```



Access the Pod's logs to see the data being written:

```
kubectl exec -it test-vol2 -- bash
```

You should see the dates being appended to the log file.

Check the host directory to verify the volume:

If you have access to the node running the Pod, you can verify the data in /tmp/data/log.txt.

Thank You